Phân tích dữ liệu bằng SQL

Cửa sổ phân tích · Xếp hạng · Tổng lũy kế · So sánh chuỗi thời gian



Môi trường: MySQL 8.0+ · Schema: `window_function_lab`

Mục tiêu học tập

- Hiểu sự khác nhau giữa `GROUP BY` và `OVER(...)`.
- Viết được `PARTITION BY`, `ORDER BY` và frame.
- Tính tổng theo nhóm, tỷ trọng, running total, moving average.
- Xếp hạng và giải bài toán top-*N* theo nhóm.
- So sánh các dòng liên tiếp bằng `LAG/LEAD`; lấy giá trị đầu/cuối.
- Sử dụng `NTILE`, `PERCENT_RANK`, `CUME_DIST`.
- Kết hợp CTE, named windows và nhận diện lỗi truy vấn thường gặp.

Lộ trình bài giảng

1. **Nền tảng:** Dữ liệu, GROUP BY, cú pháp OVER.
2. **Aggregate windows:** Tỷ trọng, running total, moving average.
3. **Ranking:** ROW_NUMBER, RANK, DENSE_RANK, top-N.
4. **Value và distribution:** LAG, LEAD, FIRST/LAST/NTH, NTILE.
5. **Nâng cao:** ROWS/RANGE, WINDOW clause, aggregate rồi window.
6. **Thực hành:** Constraints, performance, bài tập tổng hợp.

Dataset: bảng sales

```sql
CREATE TABLE sales (
  sale_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  sales_rep VARCHAR(50) NOT NULL,
  region VARCHAR(30) NOT NULL,
  product_name VARCHAR(100) NOT NULL,
  sale_date DATE NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  CHECK (amount > 0),
  INDEX idx_rep_date (sales_rep, sale_date, sale_id),
  INDEX idx_region_amount (region, amount)
);
```

Cấu trúc dữ liệu

- 1 row = 1 giao dịch bán hàng.
- Hai khu vực: **North**, **South**.
- Bốn nhân viên: **An, Binh, Chau, Dung**.
- 21 dòng, có doanh thu bằng nhau và giao dịch cùng ngày.

Dataset: các thống kê kiểm tra

Tổng doanh thu từng vùng

Region	Rows	Revenue
North	11	7,350.00
South	10	6,850.00
All	**21**	**14,200.00**

Tổng theo nhân viên

Sales rep	Revenue
An	4,200.00
Binh	3,150.00
Chau	3,400.00
Dung	3,450.00

Chú ý: Những con số minh họa trong slide được tính từ chính 21 giao dịch mẫu. Script tạo dữ liệu đi kèm tài liệu.

GROUP BY và Window Function



	GROUP BY	Window function
Đơn vị đầu ra	Mỗi nhóm	Mỗi dòng chi tiết
Kết quả	Tổng hợp	Phân tích gắn vào từng dòng
Phù hợp	Báo cáo tóm tắt	Bảng giao dịch + KPI

Ví dụ GROUP BY: chỉ lấy kết quả tổng hợp

```sql
SELECT region,
     SUM(amount) AS revenue
FROM sales
GROUP BY region;
```

Output

region	revenue
North	7,350.00
South	6,850.00

21 giao dịch được gom còn 2 rows.

Ví dụ Window SUM: giữ dữ liệu giao dịch

```sql
SELECT sale_id,
     region,
     amount,
     SUM(amount) OVER (
       PARTITION BY region
     ) AS region_total
FROM sales
ORDER BY region, sale_id;
```

Output rút gọn

sale	region	amount	total
1	North	1,200	7,350
2	North	200	7,350
12	South	1,500	6,850
13	South	200	6,850

Giữ 21 rows, thêm tổng của region tương ứng.

Cú pháp của OVER()

```
function_name(expression) OVER (
  [PARTITION BY expression [, ...]]
  [ORDER BY expression [ASC | DESC] [, ...]]
  [ROWS | RANGE frame_specification]
)
```

Thành phần	Câu hỏi
`PARTITION BY`	Chia nhóm để tính riêng?
`ORDER BY`	Thứ tự logic trong từng nhóm?
`Frame`	Tại dòng hiện tại sẽ xét những rows nào?

Không viết `PARTITION BY` $\Rightarrow$ toàn bộ input của query block thuộc một partition.

OVER() rỗng và PARTITION BY

```sql
SELECT sale_id, region, amount,
     SUM(amount) OVER ()
       AS company_total,
     SUM(amount) OVER (
       PARTITION BY region
     ) AS region_total
FROM sales;
```

Cách đọc

- `OVER()`: tổng **14,200** tại mọi row.
- `PARTITION BY region`: North = **7,350**; South = **6,850**.
- Không làm thay đổi số lượng giao dịch đầu ra.

Hai loại ORDER BY rất khác nhau

```sql
SELECT sale_id, sales_rep,
     sale_date, amount,
     ROW_NUMBER() OVER (
       PARTITION BY sales_rep
       ORDER BY sale_date, sale_id
     ) AS sale_seq
FROM sales
ORDER BY region, sales_rep,
      sale_date, sale_id;
```

Bên trong OVER(...)

Xác định **thứ tự tính toán**: row number, lag/lead, running total.

ORDER BY cuối truy vấn

Xác định **thứ tự trình bày** của result set.

Mẹo: Thêm `sale_id` để phá hòa khi cần sequence xác định.

Peer rows và giá trị đồng hạng (ties)

sale_id	amount	RANK theo amount
1	1,200	1
5	1,200	1
7	1,200	1
8	900	4

- Khi `ORDER BY amount DESC`, ba giao dịch amount = 1,200 là **peers**.
- Nếu thêm `sale_id` vào window ordering, không còn peers ở cùng amount.
- **Không thêm tie-breaker** cho `RANK/DENSE_RANK` nếu yêu cầu là cùng amount nhận cùng hạng.

Aggregate windows: thống kê theo vùng

```sql
SELECT sale_id, region, amount,
  COUNT(*) OVER (
    PARTITION BY region) AS n,
  AVG(amount) OVER (
    PARTITION BY region) AS avg_amt,
  MIN(amount) OVER (
    PARTITION BY region) AS min_amt,
  MAX(amount) OVER (
    PARTITION BY region) AS max_amt
FROM sales;
```

Kết quả theo region

	North	South
COUNT	11	10
AVG	668.18	685.00
MIN	200	200
MAX	1,200	1,500

Các KPI lặp lại trên mỗi sale thuộc cùng region.

Tính tỷ trọng mỗi giao dịch

```sql
SELECT sale_id, region, amount,
  ROUND(100.0 * amount /
    SUM(amount) OVER (
      PARTITION BY region
    ), 2) AS pct_of_region
FROM sales
ORDER BY region, sale_id;
```

Business question

Mỗi giao dịch đóng góp bao nhiêu phần trăm vào doanh thu của region?

Ví dụ:

- Sale 1: $1200/7350 \times 100 = 16.33\%$.
- Sale 12: $1500/6850 \times 100 = 21.90\%$.

Lọc theo kết quả window: cần CTE

```sql
WITH with_avg AS (
 SELECT sale_id, region, amount,
   AVG(amount) OVER (
     PARTITION BY region
   ) AS region_avg
 FROM sales
)
SELECT *
FROM with_avg
WHERE amount > region_avg;
```

Tại sao?

- `WHERE` trong cùng query block được xử lý trước window calculation.
- Không thể dùng `AVG(...) OVER(...)` trực tiếp trong `WHERE`.
- Lớp truy vấn ngoài lọc theo cột do CTE tính trước.

Running total: doanh thu tích lũy

```sql
SELECT sale_id, sales_rep,
     sale_date, amount,
 SUM(amount) OVER (
  PARTITION BY sales_rep
  ORDER BY sale_date, sale_id
  ROWS BETWEEN UNBOUNDED PRECEDING
         AND CURRENT ROW
 ) AS running_revenue
FROM sales;
```

Cơ chế

- Reset tổng khi chuyển sang nhân viên khác.
- Thứ tự tính theo ngày và sale ID.
- Frame mở từ sale đầu tiên đến sale hiện tại.
- Dùng `ROWS` để cộng theo từng record.

Running total: ví dụ của An

sale_id	amount	running_revenue
1	1,200	1,200
2	200	1,400
3	800	2,200
4	500	2,700
5	1,200	3,900
6	300	4,200

Giá trị tại sale 5 = 1,200 + 200 + 800 + 500 + 1,200 = **3,900**. Sale 6 đưa tổng lên **4,200**.

Moving average: trung bình 3 giao dịch

```sql
SELECT sale_id, sales_rep, amount,
 ROUND(AVG(amount) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
   ROWS BETWEEN 2 PRECEDING
          AND CURRENT ROW
 ), 2) AS moving_avg_3
FROM sales;
```

Đối với An

sale	amount	avg 3
1	1,200	1,200
2	200	700
3	800	733.33
4	500	500
5	1,200	833.33
6	300	666.67

Moving frame: quá khứ và tương lai

```sql
-- Current row + 2 previous sales
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

-- Previous, current, next (centered)
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

-- First to current
ROWS BETWEEN UNBOUNDED PRECEDING
      AND CURRENT ROW
```

Lưu ý về biên

Dòng đầu hoặc cuối partition có thể không đủ số lượng giao dịch trong frame.

- Ở row đầu: chỉ có current row nếu dùng two preceding.
- Với centered average, row cuối không có following row.

Ba hàm ranking quan trọng

amount ↓	ROW_NUMBER()	RANK()	DENSE_RANK()
1,500	1	1	1
1,200	2	2	2
1,200	3	2	2
900	4	4	3

- `ROW_NUMBER()`: đánh số duy nhất, không giữ ties.
- `RANK()`: đồng hạng và **bỏ số** sau ties.
- `DENSE_RANK()`: đồng hạng nhưng **không bỏ số**.

So sánh 3 hàm ranking qua SQL

```sql
SELECT sale_id, region, amount,
  ROW_NUMBER() OVER (
    PARTITION BY region
    ORDER BY amount DESC, sale_id
  ) AS row_num,
  RANK() OVER (
    PARTITION BY region
    ORDER BY amount DESC
  ) AS amount_rank,
  DENSE_RANK() OVER (
    PARTITION BY region
    ORDER BY amount DESC
  ) AS dense_rank
FROM sales
ORDER BY region, amount DESC, sale_id;
```

`sale_id` chỉ thêm vào `ROW_NUMBER` để phá ties; không thêm vào `RANK` và `DENSE_RANK` khi cần giữ đồng hạng.

Top-N theo region: đúng N dòng

```sql
WITH numbered AS (
 SELECT sale_id, region, amount,
   ROW_NUMBER() OVER (
     PARTITION BY region
     ORDER BY amount DESC, sale_id
   ) AS rn
 FROM sales
)
SELECT * FROM numbered
WHERE rn <= 2
ORDER BY region, rn;
```

Kết quả mẫu

- North: sale 1 và sale 5.
- South: sale 12 và sale 17.

Khái quát

Mỗi region được chọn **tối đa 2 rows**, kể cả khi amount trùng nhau.

Top-N: chọn đúng business semantics

Yêu cầu	Hàm + bộ lọc	Số dòng có thể trả
Đúng 2 giao dịch	`ROW_NUMBER` ≤ 2	Tối đa 2 / region
2 hạng đầu, giữ đồng hạng	`RANK` ≤ 2	Có thể > 2
2 mức amount khác nhau	`DENSE_RANK` ≤ 2	Có thể > 2

North: ba sales amount 1,200 đồng hạng 1. `RANK` tiếp theo là 4; `DENSE_RANK` tiếp theo là 2.

Quy tắc chọn: N rows $\rightarrow$ ROW_NUMBER; ranking kiểu thi đấu $\rightarrow$ RANK; N giá trị phân biệt $\rightarrow$ DENSE_RANK.

Deduplication: giữ giao dịch mới nhất

```sql
WITH numbered AS (
 SELECT sale_id, sales_rep,
      product_name, sale_date,
  ROW_NUMBER() OVER (
    PARTITION BY sales_rep, product_name
    ORDER BY sale_date DESC, sale_id DESC
  ) AS rn
 FROM sales
)
SELECT * FROM numbered WHERE rn = 1;
```

Business key:

`(sales_rep, product_name)`

- Mỗi key chọn 1 giao dịch gần nhất.
- An + Laptop: giữ sale 5 thay vì sale 1.
- Query chọn/dò duplicates; không thay thế ràng buộc `UNIQUE`.

LAG và LEAD: so sánh lân cận

```sql
SELECT sale_id, sales_rep,
     sale_date, amount,
 LAG(amount) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
 ) AS prev_amount,
 LEAD(amount) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
 ) AS next_amount
FROM sales;
```

Định nghĩa

- `LAG(expr, offset, default)`: lùi về dòng trước.
- `LEAD(expr, offset, default)`: tiến đến dòng sau.
- `offset` mặc định 1; `default` mặc định `NULL`.

Yêu cầu bắt buộc về ý nghĩa

Xác định rõ thứ tự nghiệp vụ: ngày, thời gian, số phiên bản, v.v.

LAG: tính thay đổi so với giao dịch trước

```sql
SELECT sale_id, sales_rep, amount,
     LAG(amount) OVER (
       PARTITION BY sales_rep
       ORDER BY sale_date, sale_id
     ) AS prev_amount,
     amount - LAG(amount) OVER (
       PARTITION BY sales_rep
       ORDER BY sale_date, sale_id
     ) AS change_from_previous
FROM sales;
```

An: sale	amount	previous	change
1	1,200	NULL	NULL
2	200	1,200	-1,000
3	800	200	+600

LAG và chuỗi thời gian: số ngày giữa sales

```sql
SELECT sale_id, sales_rep, sale_date,
 DATEDIFF(
  sale_date,
  LAG(sale_date) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
  )
 ) AS days_since_previous
FROM sales;
```

Kết quả của An

- Sale 1: `NULL` (không có row trước).
- Sale 2: **0 ngày** (cùng 2026-01-03).
- Sale 3: **9 ngày** (từ 03/01 đến 12/01).

Có thể dùng `LAG(sale_date, 1, ...)` nếu muốn quy định giá trị mặc định.

FIRST_VALUE, LAST_VALUE và NTH_VALUE

```
FIRST_VALUE(amount) OVER w_full
LAST_VALUE(amount) OVER w_full
NTH_VALUE(amount, 2) OVER w_full

-- Definition for the whole partition:
WINDOW w_full AS (
  PARTITION BY sales_rep
  ORDER BY sale_date, sale_id
  ROWS BETWEEN UNBOUNDED PRECEDING
         AND UNBOUNDED FOLLOWING
)
```

- `FIRST_VALUE`: amount giao dịch đầu.
- `LAST_VALUE`: amount giao dịch cuối.
- `NTH_VALUE(...,2)`: amount giao dịch thứ hai.

Bẫy thường gặp: LAST_VALUE và default frame

Dễ sai ý định

```
LAST_VALUE(amount) OVER (
  PARTITION BY sales_rep
  ORDER BY sale_date, sale_id
)
```

Với `ORDER BY`, mặc định frame thường kết thúc ở current row.

Lấy giá trị cuối toàn partition

```
LAST_VALUE(amount) OVER (
  PARTITION BY sales_rep
  ORDER BY sale_date, sale_id
  ROWS BETWEEN UNBOUNDED PRECEDING
         AND UNBOUNDED FOLLOWING
)
```

An có giá trị cuối = **300**.

NTH_VALUE: lấy vị trí thứ k

```sql
SELECT sale_id, sales_rep, amount,
 NTH_VALUE(amount, 2) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
   ROWS BETWEEN UNBOUNDED PRECEDING
          AND UNBOUNDED FOLLOWING
 ) AS second_sale_amount
FROM sales;
```

Giá trị giao dịch thứ hai:

Rep	Second amount
An	200
Binh	900
Chau	200
Dung	900

Nếu partition có dưới 2 rows, kết quả là `NULL`.

NTILE: chia dữ liệu thành buckets

```sql
SELECT sale_id, region, amount,
 NTILE(4) OVER (
   PARTITION BY region
   ORDER BY amount DESC, sale_id
 ) AS revenue_quartile
FROM sales;
```

Chia thành 4 nhóm gần đều

- North: 11 rows → **3 / 3 / 3 / 2**.
- South: 10 rows → **3 / 3 / 2 / 2**.
- Bucket 1 ở thứ tự DESC gồm các sale lớn hơn.
- Peers có thể bị tách buckets; thêm tie-breaker cho phân nhóm ổn định.

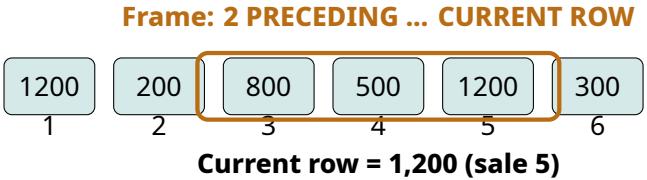
PERCENT_RANK và CUME_DIST

Hàm	Ý nghĩa	Công thức / ví dụ
`PERCENT_RANK`	Vị trí tương đối theo rank	$(rank - 1)/(n - 1)$
`CUME_DIST`	Tỷ lệ rows $\leq$ giá trị hiện tại	số rows $\leq x$ / n

```sql
SELECT region, amount,
     PERCENT_RANK() OVER (
       PARTITION BY region ORDER BY amount
     ) AS pr,
     CUME_DIST() OVER (
       PARTITION BY region ORDER BY amount
     ) AS cd
FROM sales;
```

North: amount = 1,200 (lớn nhất, 3 rows đồng hạng) có `PERCENT_RANK = 0.80` và `CUME_DIST = 1.00`.

Frame là gì?



- **Partition** là nhóm rows liên quan.
- **Order** tạo vị trí logic trong nhóm.
- **Frame** là đoạn con trong partition mà hàm đang xét.
- Sale 5: frame = 800, 500, 1,200 $\Rightarrow$ AVG = **833.33**.

ROWS và RANGE: cùng ngày, kết quả khác nhau

sale	date	amount	Default RANGE	Explicit ROWS
1	2026-01-03	1,200	1,400	1,200
2	2026-01-03	200	1,400	1,400
3	2026-01-12	800	2,200	2,200

RANGE … CURRENT ROW

Khi order theo `sale_date`, các giao dịch cùng ngày là peers; frame có thể bao gồm **cả nhóm peers**.

ROWS … CURRENT ROW

Khi order theo `sale_date, sale_id`, running total đi **từng dòng** một.

Cú pháp frame: những biên quan trọng

Boundary	Ý nghĩa
`UNBOUNDED PRECEDING`	Đầu partition
`N PRECEDING`	N vị trí / giá trị trước
`CURRENT ROW`	Dòng hiện tại (RANGE gồm peers)
`N FOLLOWING`	N vị trí / giá trị sau
`UNBOUNDED FOLLOWING`	Cuối partition

Với ordered aggregate nhưng **không khai báo frame**, mặc định MySQL sử dụng `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Các frame thường dùng

```sql
-- Running total by record
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

-- Moving average of at most 3 records
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

-- Centered 3-record window
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

-- First or last value of whole partition
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
```

Lưu ý: không phải mọi window function đều phụ thuộc frame; ví dụ ranking dùng ordering/partition, không tính trên moving frame.

Named windows: dùng lại cùng một định nghĩa

```sql
SELECT sale_id, sales_rep,
     sale_date, amount,
  ROW_NUMBER() OVER w AS sale_no,
  LAG(amount) OVER w AS prev_amount
FROM sales
WINDOW w AS (
 PARTITION BY sales_rep
 ORDER BY sale_date, sale_id
)
ORDER BY sales_rep, sale_date, sale_id;
```

Ưu điểm

- Tránh lặp định nghĩa window.
- Hạn chế lỗi thứ tự không thống nhất.
- Dễ đọc và bảo trì.
- Có thể định nghĩa nhiều windows với các frames khác nhau.

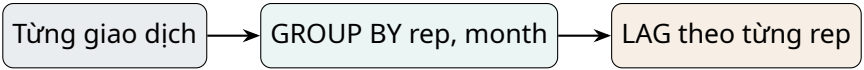
`WINDOW` đứng trước final `ORDER BY`.

Named windows: cùng order, khác frame

```sql
SELECT sale_id, sales_rep, amount,
  SUM(amount) OVER w_running AS cumulative,
  AVG(amount) OVER w_last3 AS moving_avg
FROM sales
WINDOW
 w_running AS (
   PARTITION BY sales_rep ORDER BY sale_date, sale_id
   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
 ),
 w_last3 AS (
   PARTITION BY sales_rep ORDER BY sale_date, sale_id
   ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
 );
```

Ở sale 5 của An: `cumulative = 3900`; `moving_avg = 833.33`.

Aggregate trước, window sau: đúng đơn vị phân tích




```sql
WITH monthly AS (
 SELECT sales_rep, DATE_FORMAT(sale_date, '%Y-%m-01') AS mon,
       SUM(amount) AS monthly_total
 FROM sales
 GROUP BY sales_rep, DATE_FORMAT(sale_date, '%Y-%m-01')
)
SELECT sales_rep, mon, monthly_total,
      LAG(monthly_total) OVER (
        PARTITION BY sales_rep ORDER BY mon
      ) AS prev_month
FROM monthly;
```

Monthly analytics: ví dụ của An

Month	Monthly revenue	Previous month	Change
2026-01	2,200	NULL	NULL
2026-02	1,700	2,200	-500
2026-03	300	1,700	-1,400

- Running monthly revenue: **2,200** $\rightarrow$ **3,900** $\rightarrow$ **4,200**.
- Cần phân tích theo tháng: **GROUP BY month trước**, rồi mới dùng `LAG`.
- Nếu một tháng không có giao dịch, `LAG` lấy row tháng có dữ liệu trước đó; muốn tháng liên tục phải tạo calendar/time spine.

Giới hạn của MySQL Window Functions

Dùng trực tiếp

- `SELECT` list
- `ORDER BY`

Không dùng trực tiếp

- `WHERE`, `HAVING`
- `GROUP BY`, `JOIN ... ON`
- `UPDATE ... SET` cùng query block

Một số constructs không hỗ trợ:

- Nested window functions.
- `DISTINCT` trong aggregate window.
- `GROUPS` frame, `EXCLUDE`.
- `IGNORE NULLS`.
- `NTH_VALUE ... FROM LAST`.

Sử dụng CTE/derived tables khi cần thêm lớp tính toán.

Những lỗi cần nhớ khi thực hành

Lỗi	Cách khắc phục
Dùng GROUP BY khi cần từng sale	Dùng aggregate `OVER(PARTITION BY ...)`
`ROW_NUMBER` thiếu tie-break	Thêm primary key vào window ordering
`RANK` cho exactly N rows	Dùng `ROW_NUMBER`
Running SUM mặc định dùng RANGE	Viết `ROWS` explicit
`LAST_VALUE` sai giá trị cuối	Khai báo full-partition frame
Filter rank ngay trong WHERE	Tách CTE và outer WHERE
Nhầm order trong OVER với output	Viết final ORDER BY

WHERE lọc trước window: khác biệt nghiệp vụ

```sql
-- Revenue within Feb-Mar only
SELECT region, sale_id, amount,
 SUM(amount) OVER (
   PARTITION BY region
 ) AS total_after_filter
FROM sales
WHERE sale_date >= '2026-02-01';
```

Kết quả

- North: **3,050** (Feb–Mar).
- South: **2,750** (Feb–Mar).

Nếu muốn **tổng cả lịch sử** nhưng chỉ hiển thị sale từ tháng 2: tính window trong CTE trước, rồi filter ngày ở outer query.

Performance: kiểm tra thay vì suy đoán

```
EXPLAIN
SELECT sale_id, sales_rep,
 SUM(amount) OVER (
   PARTITION BY sales_rep
   ORDER BY sale_date, sale_id
   ROWS BETWEEN UNBOUNDED PRECEDING
          AND CURRENT ROW
 ) AS running_revenue
FROM sales;
```

Chi phí phụ thuộc:

- Số rows và số partitions.
- Sort, memory và temporary data.
- Điều kiện lọc, indexes và query plan.
- Server version và configuration.

Index khớp với partition/order có thể hữu ích, **không đảm bảo** loại bỏ mọi sort.

Mở rộng classicmodels: customer payments

```sql
SELECT customerNumber, paymentDate, checkNumber, amount,
 SUM(amount) OVER (
   PARTITION BY customerNumber
   ORDER BY paymentDate, checkNumber
   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
 ) AS running_paid_amount,
 LAG(amount) OVER (
   PARTITION BY customerNumber
   ORDER BY paymentDate, checkNumber
 ) AS previous_payment
FROM classicmodels.payments
ORDER BY customerNumber, paymentDate, checkNumber;
```

Chỉ chạy nếu đã cài `classicmodels`. Sử dụng `checkNumber` để phá hòa khi một customer có nhiều payment cùng ngày.

Thực hành A: window aggregate và frame

1. Hiển thị `sale_id, region, sales_rep, amount` kèm **company total**, **region total** và **percentage in region**.
2. Với mỗi nhân viên, tính **running revenue** theo `sale_date, sale_id`.
3. Với mỗi nhân viên, tính **moving average** của tối đa 3 giao dịch gần nhất.
4. So sánh hai running totals cho An: dùng default `RANGE` và explicit `ROWS`.
5. Lọc ra sales có `amount` lớn hơn average amount của `region` tương ứng.

Thực hành B: ranking và value functions

1. Viết 3 truy vấn lấy top 2 sale theo mỗi region: **exactly 2 rows**, **giữ ties**, **2 amount levels**.
2. Ghi thứ tự `ROW_NUMBER`, `RANK`, `DENSE_RANK` cho dãy: **100, 100, 80, 50**.
3. Với mỗi sale, tính amount trước, amount sau và chênh lệch amount trước.
4. Hiển thị first sale, last sale và second sale amount theo nhân viên.
5. Chia sales của từng region thành 4 nhóm bằng `NTILE(4)`; giải thích ties.

Mini-project 1: Sales dashboard

Viết một query trên bảng `sales` để trả về:

1. `sale_id, region, sales_rep, sale_date, amount`.
2. Tổng doanh thu **toàn công ty** và **theo region**.
3. Mức doanh thu **trung bình của region**.
4. Tỷ trọng **% của sale** trong region.
5. `RANK()` giữ ties theo amount và `ROW_NUMBER()` deterministic.
6. Sắp đầu ra theo region, rank và sale ID.

Kiểm thử: company total = 14,200; North = 7,350; South = 6,850.

Mini-project 2: Rep performance analytics

Đầu ra cho từng nhân viên, theo trình tự thời gian:

- Số thứ tự giao dịch: `ROW_NUMBER()`.
- Giao dịch trước và sau: `LAG()`, `LEAD()`.
- Thay đổi so với giao dịch trước.
- Tổng lũy kế: `SUM() OVER(... ROWS ...)`.
- Moving average của 3 giao dịch.
- Amount của giao dịch đầu và cuối toàn partition.

Gợi ý: Định nghĩa các named windows `w_order`, `w_running`, `w_last3`, `w_full`.

Mini-project 3: Monthly sales analytics

1. Tạo CTE doanh thu theo `region` và `month`.
2. Tính doanh thu tháng trước bằng `LAG`.
3. Tính **month-over-month change** và running monthly revenue.
4. Chỉ lấy các tháng có doanh thu tăng so với tháng trước (outer query).
5. Thảo luận tháng không có doanh thu: `LAG` đang so sánh với tháng lịch liền trước hay row trước đó?

Mục tiêu cốt lõi: Chọn đúng *grain* (đơn vị phân tích), sau đó mới chọn window function.

Câu hỏi nhanh kiểm tra kiến thức

1. `SUM(amount) OVER(PARTITION BY region)` có làm giảm số rows không?
2. Muốn lấy đúng 3 dòng theo mỗi region dùng `RANK` hay `ROW_NUMBER`?
3. Để running total đi từng giao dịch khi có nhiều giao dịch cùng ngày, nên dùng `ROWS` hay `RANGE`?
4. Vì sao `LAST_VALUE` cần full frame để lấy giao dịch cuối cùng?
5. Muốn lọc `row_num <= 3`, vì sao cần thêm query layer?

Đáp án kiểm tra nhanh

1. **Không.** Window aggregate giữ các detail rows.
2. **ROW_NUMBER.** Có tie-breaker để chọn tối đa 3 dòng.
3. **ROWS** với thứ tự xác định, ví dụ `sale_date, sale_id`.
4. Default frame của ordered window thường chỉ đến current row, chưa đến cuối partition.
5. Window calculation không dùng trực tiếp trong `WHERE` cùng query block; tính trong CTE rồi lọc ở query ngoài.

Tổng kết: chọn window function theo nhu cầu

Câu hỏi phân tích	Công cụ chính
Tổng/avg theo nhóm, giữ detail	`SUM/AVG(...) OVER(PARTITION BY ...)`
Doanh thu tích lũy / rolling avg	Aggregate window + explicit `ROWS`
Đánh số, ranking, top-N	`ROW_NUMBER/RANK/DENSE_RANK`
So sánh các thời điểm liên tiếp	`LAG/LEAD`
Giá trị đầu/cuối/thứ N	`FIRST/LAST/NTH_VALUE`
Phân vị, vị trí trong phân phối	`NTILE/PERCENT_RANK/CUME_DIST`
Loại bỏ theo kết quả window	`CTE` / derived table

Tài liệu và hướng dẫn thực hành

- **Bài giảng gốc:** DBMS Course – MySQL Window Functions.
- **MySQL Reference Manual:** Window Function Descriptions.
- **Cú pháp và frame:** Window Function Frame Specification.
- **Chuẩn bị:** MySQL 8.0+, schema `window_function_lab`; có thể mở rộng với `classicmodels`.
- **File đi kèm:** `window_functions_lab.sql` cung cấp dữ liệu mẫu và kiểm thử.

Cảm ơn!